

MiniDB: gestor de bases de datos relacional en disco

Renzo Acervo^a, Josué Toribio^a, Lisseth Rondan^a

^a*Departamento de Ciencia de la Computación, Universidad de Ingeniería y Tecnología (UTEC), Lima, Perú*

Proyecto 1 (Entregable 1) — Base de Datos II (CS2042) — 2026-II — Docente: Percy Lovon Ramos

Resumen

MiniDB es un gestor de bases de datos relacional construido desde cero en Python que opera directamente sobre memoria secundaria. Todo dato e índice vive en archivos binarios organizados en páginas de tamaño fijo que se transfieren de a una con `seek + read/write` y se serializan con `struct`. Implementamos un Heap File con *free-list*, un Sequential File con área de overflow encadenada y reorganización con *fill factor*, un árbol B^+ multinivel y un Extendible Hashing, ambos en disco; un motor SQL propio (lexer, parser de descenso recursivo, planificador con estimación de costos y ejecutor), una API REST y un cliente web de cuatro paneles. Un monitor `DiskCounter` registra exactamente las páginas leídas y escritas por cada sentencia. Con 100 000–500 000 registros sintéticos, los cuatro experimentos pedidos reproducen los costos teóricos: la inserción en el Heap cuesta 2 I/O, la búsqueda puntual cuesta P lecturas con full scan, $\lceil \log_2 M \rceil$ con búsqueda binaria, $h + 1$ con el B^+ y 3 con el hash, y un índice no agrupado deja de convenir frente al full scan cuando el rango supera $\approx P/N$ de la tabla.

Palabras clave: Bases de datos, Memoria secundaria, Paginación, Heap File, Sequential File, Árbol B^+ , Extendible Hashing, Costo de I/O.

Índice

1. Introducción	3
2. Arquitectura del sistema	3
3. Diseño físico	4
3.1. DiskManager y DiskCounter	4
3.2. Header común de página	4
3.3. Registros de longitud fija con bitmap y RID	5
3.4. Catálogo	5
4. Organización de archivos	5
4.1. Heap File con Free-List	5
4.2. Sequential File	6
5. Índices	7
5.1. Árbol B ⁺ en disco	7
5.2. Extendible Hashing en disco	7
6. Motor SQL	8
6.1. Lexer y parser	8
6.2. Planificador	8
6.3. Ejecutor	9
7. API REST y cliente web	9
8. Análisis de costos teóricos	11
9. Experimentación	11
9.1. Metodología	11
9.2. Experimento 1: costo de inserción masiva	11
9.3. Experimento 2: búsquedas puntuales de igualdad	14
9.4. Experimento 3: búsquedas por rango con selectividad variable	15
9.5. Experimento 4: sensibilidad al tamaño de bloque	16
10.Pruebas	18
11.Conclusiones	18

1. Introducción

El cuello de botella de un gestor de bases de datos es la transferencia de bloques entre disco y memoria, por lo que el costo de cada operación se mide en páginas transferidas. El objetivo de este entregable es construir el núcleo relacional de un gestor que haga visible ese costo: cada estructura se diseña para minimizar el número de accesos a disco y cada consulta informa exactamente cuántos hizo.

Alcance. El sistema implementa todos los componentes del Entregable 1: almacenamiento paginado con `DiskCounter`; Heap File y Sequential File; árbol B⁺ y hashing dinámico en disco; SQL con `CREATE TABLE`, `INSERT`, `SELECT ... WHERE`, `DELETE` y `CREATE INDEX` (más `DROP`, `COPY` y `EXPLAIN`); planificador con desglose de I/O y latencia; API REST; cliente web y experimentación con hasta 500 000 registros.

Restricciones respetadas. No se usa ningún motor de bases de datos ni ORM; ningún archivo se serializa con `pickle`, `json` o similares (todo es `struct` little-endian); nunca se lee un archivo completo: solo `f.seek(offset) + f.read(page_size)`; no hay `mmap` y todo acceso a disco pasa por `DiskManager`; la función hash persistida es `zlib.crc32` (no `hash()`, que en Python está salteado por proceso); y ninguna operación carga una tabla completa en memoria (ni siquiera `reorganize`, que usa ordenamiento externo si hace falta).

Tecnologías. Backend en Python 3.12 con FastAPI; frontend en React + TypeScript (Vite) con CodeMirror 6 y Recharts; todo se levanta con `docker compose up -build`. Los benchmarks usan matplotlib solo para graficar.

2. Arquitectura del sistema

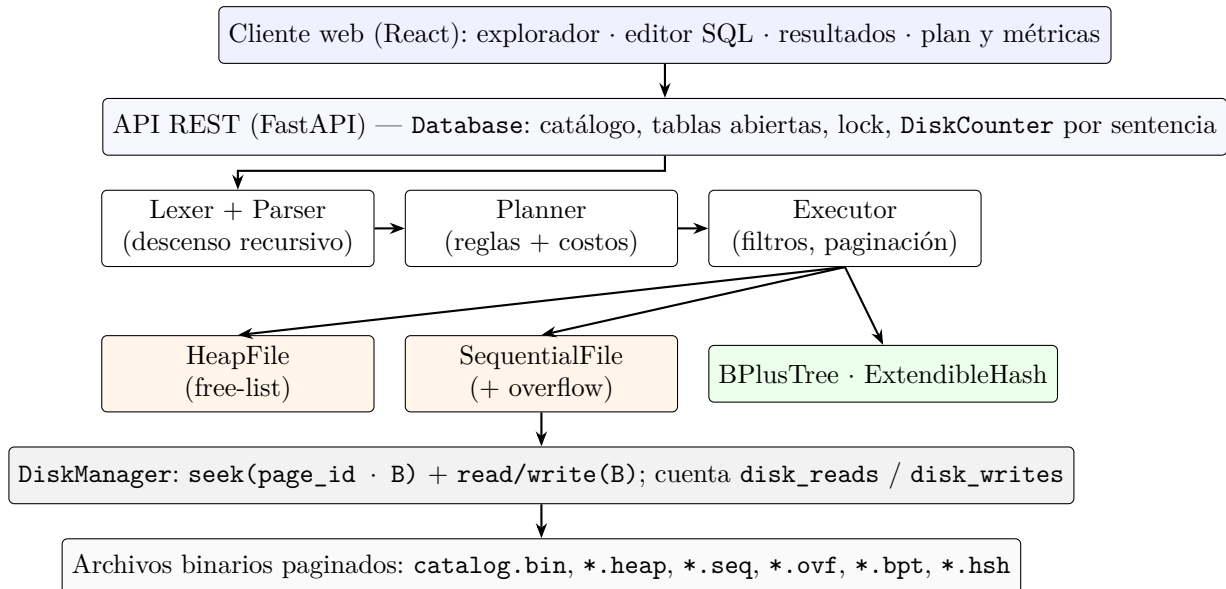


Figura 1: Capas de MiniDB. Todo acceso a disco pasa por `DiskManager`, que cuenta cada página transferida.

Una consulta sigue el camino de la figura 1: el cliente envía el SQL a `POST /api/query`; `Database.execute` lo parsea sentencia por sentencia (el lexer produce tokens de forma perezosa, así el tiempo de parseo de cada sentencia se mide por separado), resetea el `DiskCounter` compartido, ejecuta la sentencia, persiste las páginas 0 que cambiaron y devuelve filas paginadas,

el plan y las métricas (`disk_reads`, `disk_writes`, `parse_ms`, `exec_ms`, `total_ms`, medidos con `time.perf_counter()`). Un *lock* serializa las sentencias: el motor atiende una a la vez, de modo que el conteo de cada una es exacto.

La clase `Table` agrupa el archivo de datos (Heap o Sequential) y sus índices, y mantiene la consistencia entre ellos: todo `INSERT` y `DELETE` actualiza todos los índices y, tras una reorganización del Sequential, los índices se reconstruyen porque los RID cambian.

3. Diseño físico

3.1. DiskManager y DiskCounter

Cada archivo es un `DiskManager` que lo abre una sola vez en modo `r+b` y **sin buffer de Python** (`buffering=0`): cada `read_page(i)` es exactamente un `seek(i·B)` y un `read(B)`, y cada `write_page` un `seek` y un `write` de la página completa (se valida que `len(data) == B`). No hay *buffer pool*: así el número de páginas transferidas coincide con el costo teórico de cada algoritmo. `allocate_page()` solo reserva el siguiente `page_id` (sin I/O); la escritura del contenido inicial es la que se cuenta, de modo que agregar una página cuesta una escritura. El tamaño de página B es un parámetro **por archivo** (1024 a 8192 bytes en el Experimento 4, 4096 por defecto) y se guarda en la página 0.

El `DiskCounter` tiene exactamente las dos variables pedidas, `disk_reads` y `disk_writes`; se comparte entre todos los archivos de una ejecución y se resetea antes de cada sentencia.

3.2. Header común de página

Todas las páginas de todos los archivos empiezan con el mismo header de 24 bytes, empaquetado como `'<IBBHHiiixx'` (tabla 1).

Campo	Offset	Bytes	Tipo	Uso
<code>page_id</code>	0	4	u32	identificador del bloque dentro del archivo
<code>page_type</code>	4	1	u8	0 META, 1 HEAP, 2 SEQ_MAIN, 3 SEQ_OVERFLOW, 4 B ⁺ interno, 5 B ⁺ hoja, 6 HASH_DIR, 7 HASH_BUCKET, 8 CATALOG, 9 SORT_RUN
<code>flags</code>	5	1	u8	específico del tipo (profundidad local del bucket)
<code>record_count</code>	6	2	u16	registros o claves activos en la página
<code>free_space_offset</code>	8	2	u16	primer slot libre (0xFFFF = llena) o fin de los datos usados
<code>next_page_id</code>	10	4	i32	encadenamiento (free-list, overflow, <code>next_leaf</code>); -1 = nulo
<code>prev_page_id</code>	14	4	i32	encadenamiento (<code>prev_leaf</code> , página principal anterior)
<code>aux_page_id</code>	18	4	i32	específico (cabeza del overflow de una página principal)
relleno	22	2	–	alineación a 24 bytes

Tabla 1: Header de página (24 bytes, little-endian).

Página 0 de metadatos. Tras el header, la página 0 de cada archivo guarda `magic` (MDB1), versión, tipo de archivo, `page_size` y `num_pages` (16 bytes) y luego los campos propios de cada estructura: cabeza de la free-list y número de registros (Heap); capacidad, columna clave, fill factor y mínimo/máximo de la PK (Sequential); raíz, altura, capacidades y número de hojas (B⁺); pro-

fundidad global y la tabla de segmentos del directorio (hash). La página 0 se lee una vez al abrir el archivo y queda en memoria. Al final de cada sentencia se escribe **solo si cambió su estructura** (cabeza de la free-list, raíz o altura del B^+ , directorio del hash); las estadísticas (número de registros, mínimo y máximo de la clave) viajan en esa escritura o al cerrar el archivo. Tanto la lectura al abrir como estas escrituras se cuentan en el **DiskCounter**: por ejemplo, el **INSERT** que obliga a crear una página nueva del Heap cuesta 1W de la página nueva más 1W de la página 0 (cambió la cabeza de la free-list), mientras que un **INSERT** en una página que ya tenía espacio cuesta exactamente 1R + 1W.

3.3. Registros de longitud fija con bitmap y RID

Elegimos registros de longitud fija con bitmap de presencia. Los tipos se empaquetan con **struct**: **INT** → 'i' (4 B), **FLOAT** → 'd' (8 B) y **CHAR(n)/VARCHAR(n)** → 'ns' (UTF-8 relleno con \x00; se trunca por bytes sin cortar un carácter multibyte). La tabla del enunciado, **empleados(id INT, nombre CHAR(30), dept CHAR(20), salario FLOAT)**, se empaqueta como '**<i30s20sd**' = 62 bytes. Una página de datos tiene el layout

$$\underbrace{\text{header}}_{24 \text{ B}} \mid \underbrace{\text{bitmap}}_{\lceil \text{cap}/8 \rceil \text{ B}} \mid \underbrace{\text{slot}_0 \cdots \text{slot}_{\text{cap}-1}}_{\text{cap} \times 62 \text{ B}} \quad \text{con} \quad \text{cap} = \max\{n : 24 + \lceil n/8 \rceil + 62n \leq B\}.$$

B (bytes)	1024	2048	4096	8192
registros de 62 B por página (cap)	16	32	65	131
bytes de bitmap	2	4	9	17
bytes usados / libres	1018 / 6	2012 / 36	4063 / 33	8163 / 29
páginas para 100 000 registros (P)	6250	3125	1539	764

Tabla 2: Capacidad de las páginas de datos para la tabla **empleados**.

Cada tupla se identifica con **RID** = (**page_id**, **slot**), empaquetado como '**<iH**' (6 bytes). En el Sequential File un **page_id** negativo apunta al área de overflow: **RID**($-k$, **s**) es el slot **s** de la página k del archivo **.ovf** (la página 0 es de metadatos, así que no hay ambigüedad).

3.4. Catálogo

El catálogo (**catalog.bin**) guarda tablas, columnas y tipos, PK, organización, archivos, **page_size**, parámetros del Sequential e índices (nombre, columna, tipo y archivo). Se serializa con **struct** (cadenas con prefijo de longitud) y se reparte en páginas **CATALOG** encadenadas; la página 0 guarda la longitud total. Cada DDL lo reescribe (es pequeño) y esas escrituras se cuentan en la sentencia. Al arrancar, el backend lee el catálogo de **MINIDB_DATA_DIR** y reabre todas las tablas, de modo que todo sobrevive a un reinicio.

4. Organización de archivos

4.1. Heap File con Free-List

El Heap (**<tabla>.heap**) mantiene una lista simplemente enlazada (por **next_page_id**) de las páginas que tienen al menos un slot libre; la cabeza vive en la página 0. Invariante: una página está en la lista si y solo si tiene espacio, y siempre se inserta en la cabeza.

- **Inserción** $O(1)$: si la lista no está vacía se lee la cabeza, se ocupa el slot indicado por **free_space_offset** y se escribe (1R + 1W); si la página quedó llena sale de la lista (la cabeza pasa a su siguiente). Si la lista está vacía se agrega una página al final (1W).

- **Búsqueda:** full table scan de las páginas $1..P$, P lecturas.
- **Eliminación:** borrado lógico (se limpia el bit). Si la página estaba llena vuelve a la cabeza de la free-list. Los RID del resto no cambian. Un **DELETE** con full scan borra en el mismo recorrido (P lecturas + 1 escritura por página modificada) y uno por índice agrupa los RID por página ($1R + 1W$ por página distinta).

4.2. Sequential File

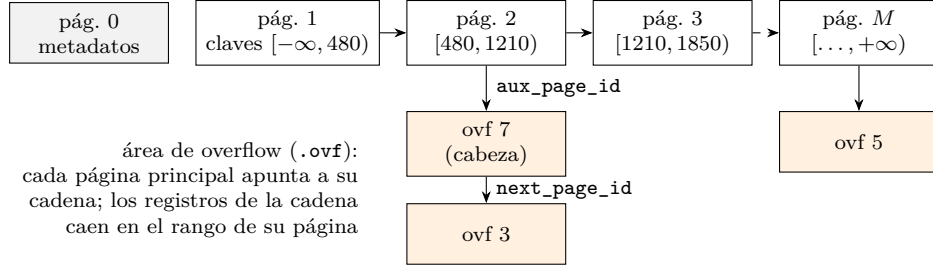


Figura 2: Sequential File: área principal ordenada por rango de PK y cadenas de overflow por página.

Estructura dual. El área principal (`<tabla>.seq`) tiene páginas `SEQ_MAIN` ordenadas por rango de clave primaria: toda clave de la página i es menor que la *clave frontera* de la página $i + 1$, lo que permite **búsqueda binaria sobre páginas**. Tras cada reorganización los registros de una página quedan ordenados en los slots $0..k - 1$, así que el slot 0 contiene la frontera. Para que esa frontera sea estable, en el área principal los slots borrados no se reutilizan hasta la siguiente reorganización (quedan como lápidas; `free_space_offset` es una marca de agua). El área de overflow (`<tabla>.ovf`) guarda las tuplas que ya no entran en su bloque: cada página principal apunta con `aux_page_id` a la cabeza de su cadena y las páginas de overflow se encadenan con `next_page_id`, manteniendo el orden lógico con punteros binarios (figura 2).

Búsqueda binaria. Se busca la última página cuya frontera es $\leq k$ leyendo solo las páginas que sondea la bisección: $\lceil \log_2 M \rceil$ lecturas (+1 si nunca se sondeó la página 1), más la cadena de overflow de esa página si la clave no está en el área principal.

Inserción. Búsqueda binaria de la página destino. Si le quedan slots vírgenes (los que dejó el fill factor) se inserta ahí sin mover otros registros (1W). Si no, va a la cabeza de su cadena de overflow (1R + 1W) o a una página de overflow nueva que pasa a ser la cabeza (1W + 1W de la página principal). Si la clave es mayor que todas las de la última página y esta no tiene overflow, se agrega una página principal nueva: inserciones en orden creciente nunca generan overflow. En el motor SQL la unicidad de la PK se valida buscando la clave en la página destino y su cadena.

Lectura ordenada. Una página lógica (principal + su cadena) se ordena en memoria antes de entregarla, sin I/O extra; así **SELECT** y los rangos salen ordenados por PK.

Reorganización. `reorganize` recorre las páginas principales en orden, fusiona cada una con su cadena de overflow y reescribe un archivo principal nuevo con **fill factor 0.75** ($\lfloor 0.75 \cdot cap \rfloor$ registros por página, configurable 0.70–0.80), en streaming: en memoria hay a lo sumo una página lógica. Si una cadena supera 64 páginas se usa **ordenamiento externo** (runs ordenados de 64 páginas en un archivo temporal y fusión con un heap de una página por run). Al final el archivo nuevo reemplaza al viejo con `os.replace`, el overflow se vacía y los índices de la tabla se reconstruyen (el de la PK con carga masiva ordenada). Se dispara automáticamente cuando las páginas de overflow superan el 10 % de las principales (desactivable, para el Experimento 1) y manualmente con **POST**

/api/tables/reorganize. Como el archivo crece geométricamente entre reorganizaciones, su costo amortizado por inserción es $O(1)$.

5. Índices

Ambos índices son **no agrupados**: guardan pares $\langle \text{clave}, \text{RID} \rangle$ y se pueden crear sobre cualquier columna de una tabla Heap o Sequential. Un índice sobre la PK es **único**: rechaza claves repetidas durante el mismo descenso de la inserción (sin lecturas extra), y así valida la PK de las tablas Heap.

5.1. Árbol B^+ en disco

B (bytes)	1024	2048	4096	8192
hoja: pares $\langle \text{INT}, \text{RID} \rangle$ ($L = \lfloor (B - 24)/10 \rfloor$)	100	202	407	816
interno: claves ($m = \lfloor (B - 28)/8 \rfloor$)	124	252	508	1020
fan-out ($m + 1$)	125	253	509	1021

Tabla 3: Capacidades del B^+ con clave INT, calculadas desde B (nunca fijas en el código).

- **Nodo interno** (BTREE_INTERNAL): $\langle P_0, K_1, P_1, \dots, K_m, P_m \rangle$ empaquetado como '*i*' + (kfmt + '*i*') $\cdot m$; `record_count` = m .
- **Hoja** (BTREE_LEAF): pares ordenados $\langle K_i, \text{RID}_i \rangle$; en el header, `next_page_id` y `prev_page_id` hacen de `next_leaf` y `prev_leaf`.
- **Inserción**: descenso guardando el camino en una pila (no hay punteros al padre); si la hoja desborda se divide al 50 % (la clave frontera se *copia* al padre), si un interno desborda se divide al 50 % (la clave del medio *sube*), y si la raíz desborda se crea una nueva raíz.
- **Búsqueda puntual**: h lecturas hasta la hoja (+1 para traer el registro por RID).
- **Rango**: descenso a la primera hoja con $K \geq lo$ y recorrido por `next_leaf`; si la cota superior del separador ya supera hi no se lee la hoja siguiente.
- **Duplicados**: se baja a la hoja más a la izquierda con `bisect_left` y se avanza a la derecha; en un índice único se baja con `bisect_right`, lo que garantiza exactamente h lecturas aunque la clave coincida con un separador.
- **Borrado**: se quita la entrada de la hoja sin fusionar ni redistribuir nodos (decisión de diseño: los separadores siguen siendo cotas válidas y el costo es $h + 1W$).
- **Carga masiva**: si la entrada está ordenada (recorrido del Sequential por PK) el árbol se construye de abajo hacia arriba con hojas llenas, escribiendo cada página una sola vez.

5.2. Extendible Hashing en disco

- **Directorio**: 2^{gd} punteros i32 a buckets en páginas `HASH_DIR` de $E = \lfloor (B - 24)/4 \rfloor$ entradas (1018 con $B = 4096$). Las páginas del directorio se reservan en *segmentos contiguos* (uno por duplicación que necesita más páginas) y la tabla de segmentos va en la página 0: ubicar la entrada i no cuesta I/O y leerla cuesta una lectura.
- **Bucket** (HASH_BUCKET): profundidad local en `flags`, pares $\langle \text{clave}, \text{RID} \rangle$ contiguos ($L = \lfloor (B - 24)/(k + 6) \rfloor$, 407 con INT y $B = 4096$) y `next_page_id` hacia su bucket de overflow.
- **Función hash**: `zlib.crc32` sobre los bytes empaquetados de la clave; se usan los gd bits bajos.

- **Split:** si el bucket lleno tiene profundidad local $< gd$ se divide por el bit ld y se redirigen las entradas del directorio correspondientes; si es igual a gd primero se duplica el directorio (en streaming, página por página). Si se llega a $gd = 20$ o todas las claves del bucket tienen el mismo hash (p.ej. valores repetidos de una columna no única) se encadena un bucket de overflow.
- **Búsqueda:** 1 lectura del directorio + 1 del bucket (+ su overflow). No soporta rangos.
- **Borrado:** se quita el par moviendo el último del bucket a su lugar, sin fusionar buckets.

6. Motor SQL

6.1. Lexer y parser

El lexer (propio) reconoce palabras clave sin distinguir mayúsculas, identificadores, enteros, reales (con exponente), strings entre comillas simples (" escapa una comilla), operadores y comentarios, y guarda la línea, columna y offset de cada token. El parser es de descenso recursivo y produce un AST (CreateTable, CreateIndex, Insert, Select, Delete, DropTable, DropIndex, Copy, Explain). Los errores indican qué se esperaba y dónde: `SELECT * FORM t` produce “se esperaba FROM pero se encontró 'FORM' (línea 1, columna 10)” y la API responde HTTP 400 con esa posición, que el editor resalta.

```
CREATE TABLE empleados (id INT PRIMARY KEY, nombre CHAR(30), dept CHAR(20),
                        salario FLOAT) USING SEQUENTIAL;
INSERT INTO empleados VALUES (101, 'Ada_Lovelace', 'Analytics', 5200.0);
CREATE INDEX idx_emp_id ON empleados (id) USING HASH;
SELECT * FROM empleados WHERE id = 101;
SELECT nombre, salario FROM empleados WHERE id BETWEEN 100 AND 500 AND dept = 'TI';
DELETE FROM empleados WHERE salario < 1500;
COPY empleados FROM 'empleados.csv'; -- carga masiva desde MINIDB_DATA_DIR
EXPLAIN SELECT * FROM empleados WHERE id >= 100 AND id <= 500;
```

6.2. Planificador

El planificador liga los predicados del WHERE al esquema (valida tipos y lleva cada literal al dominio de su columna), combina las cotas de rango por columna (dos cotas unidas con AND o un BETWEEN) y genera los caminos candidatos según las reglas del enunciado:

1. **Igualdad** `col = v`: `IndexScan` si hay un índice HASH o BTREE sobre la columna; `BinarySearch` si la tabla es SEQUENTIAL y `col` es su PK.
2. **Rango**: `IndexRangeScan` si hay un BTREE sobre la columna; `SeqFileRangeScan` si la tabla es SEQUENTIAL y `col` es su PK.
3. En cualquier otro caso, `SeqScan`.

En el modo por defecto (`rules`) se toma el primer grupo con candidatos (igualdad, luego rango, luego full scan) y, dentro del grupo, el de menor costo estimado (p.ej. HASH frente a BTREE para la misma igualdad). En el modo `cost` se elige el de menor costo entre todos, incluido el full scan: así el optimizador descarta un `IndexRangeScan` no agrupado cuando el rango es tan grande que leer toda la tabla es más barato (sección 9.4). Los predicados que no usa el acceso se aplican como filtro residual. Cada plan informa su costo estimado junto al real medido, y la lista de candidatos descartados con sus costos.

Los costos estimados son los de la tabla 5. La cantidad de registros que califican se estima con la selectividad: 1 para igualdad sobre una columna única, 1/10 para igualdad sobre una columna no

única, e interpolación lineal con el mínimo y máximo de la clave (que el B⁺ y el Sequential guardan en su página 0) para rangos numéricos.

6.3. Ejecutor

El ejecutor aplica el camino elegido. Para los full scans compila el **WHERE** en un filtro sobre los bytes crudos del registro (desempaqueta solo las columnas del predicado con **struct**; los **CHAR** se comparan como bytes, que respetan el orden de los strings), de modo que solo se decodifican las filas que califican. Los **SELECT** se ejecutan completos y se devuelve la página pedida (**page**, **page_size**) junto con el total de filas. **INSERT** valida tipos y mantiene todos los índices (si un índice único detecta un duplicado se deshace la inserción); **DELETE** usa el mismo planificador para encontrar los RID; **COPY** carga un CSV fila por fila por el mismo camino que **INSERT** (solo lee archivos dentro de **MINIDB_DATA_DIR**).

7. API REST y cliente web

Endpoint	Respuesta
POST /api/query	{ sql , page , page_size , planner } → columnas, filas de la página pedida, total, mensaje, plan (tipo, estructura, índice, predicado, filtro, costo estimado, candidatos) y métricas (disk_reads , disk_writes , parse_ms , exec_ms , total_ms). Errores 400 con tipo, mensaje y posición.
GET /api/tables	tablas con columnas y tipos, organización, número de registros y de páginas, índices (tipo, columna, altura del B ⁺ o profundidad global del hash).
POST /api/tables/reorganize	reorganiza una tabla SEQUENTIAL y devuelve páginas antes/después e I/O.
GET /api/tables/{t}/pages/{id}	header decodificado, contenido de la página y hexdump (inspector del video).

Tabla 4: Endpoints principales (contrato completo en `docs/api.md`).

El cliente web muestra a la vez los cuatro paneles pedidos: **explorador de tablas** (columnas y tipos, organización, estadísticas e índices activos, botón para reorganizar y para inspeccionar páginas), **editor SQL** con resaltado (CodeMirror 6), botón Ejecutar, atajo Ctrl+Enter y un selector del modo del planificador (reglas o costo), **visor de resultados** paginado contra el backend y **plan de ejecución y métricas** (tipo de plan, índice, lecturas y escrituras, tiempos de parseo y ejecución, costo estimado frente al real, candidatos y un gráfico del historial de I/O de las últimas consultas).

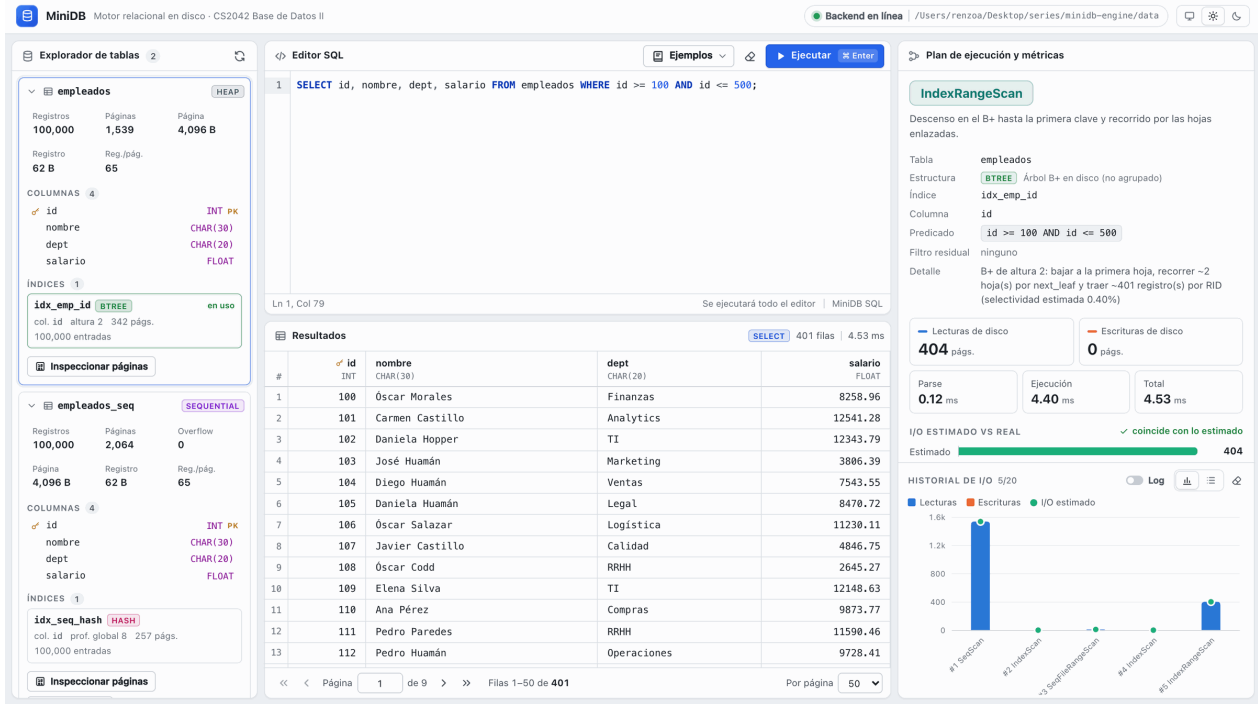


Figura 3: Cliente web con los cuatro paneles: explorador (tabla HEAP con B⁺ y tabla SEQUENTIAL con hash), editor SQL, resultados paginados (401 filas) y plan IndexRangeScan con 404 lecturas, igual al costo estimado. El historial compara las últimas consultas: SeqScan (1539 lecturas), IndexScan (3), SeqFileRangeScan (13) e IndexRangeScan (404).

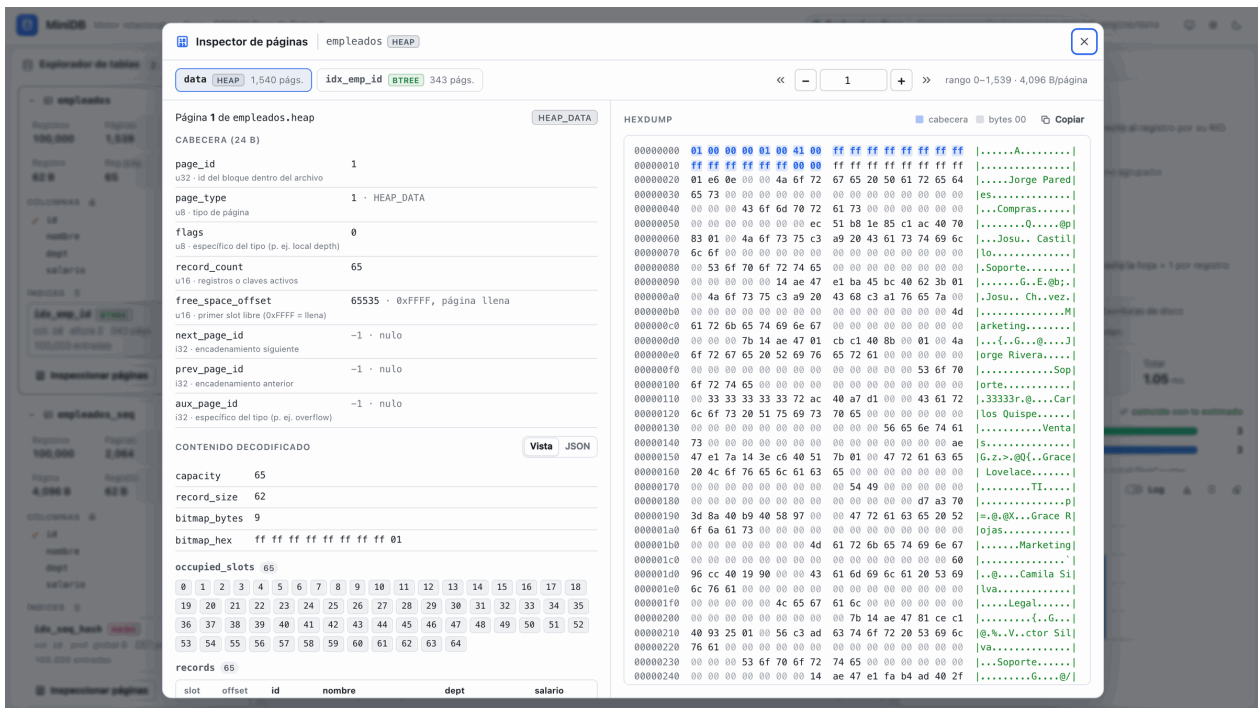


Figura 4: Inspector de páginas (GET /api/tables/{t}/pages/{id}): página 1 del Heap. En el hexdump se ven los 24 bytes del header (page_id=1, tipo 1, record_count = 41 00 = 65, free_space_offset = ff ff ff porque está llena y los punteros en -1), el bitmap (ff×8 + 01 = 65 slots ocupados) y el primer registro: e6 0e 00 00 = id 3814 seguido del nombre en UTF-8.

8. Análisis de costos teóricos

Estructura	Inserción	Búsqueda puntual	Rango (k registros)	Eliminación
Heap (free-list)	1R + 1W	P (full scan)	P	búsqueda + 1W
Sequential	$\lceil \log_2 M \rceil + 1W$ (+ overflow)	$\lceil \log_2 M \rceil + \text{ovf.}$	$\lceil \log_2 M \rceil + \lceil k/r \rceil$	búsqueda + 1W
B ⁺ (índice)	$hR + 1W$ (+ split)	h (+1 registro)	$h + \lceil k/f \rceil - 1 + k$	$h + 1W$
Hash extensible	2R + 1W (+ split)	2 (+1 registro)	no soporta	2R + 1W

Tabla 5: Costos en páginas transferidas. P : páginas de datos; M : páginas principales del Sequential; r : registros por página tras reorganizar ($\lfloor 0.75 \text{ cap} \rfloor$); h : altura del B⁺; f : entradas promedio por hoja; k : registros que califican.

La altura del B⁺ con N entradas, hojas de capacidad L y fan-out F está acotada por

$$1 + \lceil \log_F \lceil N/L \rceil \rceil \leq h \leq 1 + \lceil \log_{\lceil F/2 \rceil} \lceil N/\lfloor L/2 \rfloor \rceil \rceil,$$

ya que el split al 50 % garantiza nodos al menos a la mitad. El índice es no agrupado, por lo que un rango paga una lectura aleatoria por registro: frente al full scan (P lecturas) conviene solo si $k \lesssim P$, es decir, con selectividad menor que $P/N \approx 1/\text{cap}$ (1.5 % con $B = 4096$). El Sequential, en cambio, está agrupado por la PK: un rango lee páginas contiguas y cuesta $\approx k/r$ páginas. En el hash el costo de búsqueda no depende de N mientras no haya overflow.

9. Experimentación

9.1. Metodología

Los cuatro experimentos están automatizados en `benchmarks/` (`python -m benchmarks.run_all`) y guardan sus resultados en `benchmarks/results/` (CSV + PNG). Cada experimento crea archivos nuevos en una carpeta de trabajo, usa el dataset sintético `empleados` generado con semilla fija (ids únicos 1.. N en orden aleatorio, registros de 62 bytes) y mide con contadores independientes (`DiskCounter`) y `time.perf_counter()`, con el recolector de basura desactivado durante las mediciones. Entorno de la corrida: Python 3.12.13 en macOS-26.3.1-arm64-arm-64bit (10 núcleos), corrida del 2026-09-22 20:59:05 .

Las lecturas y escrituras contadas son **deterministas** (no dependen de la máquina) y son la métrica principal. Los tiempos son secundarios: los archivos caben en la caché de páginas del sistema operativo, de modo que una “lectura” cuesta microsegundos y el tiempo refleja sobre todo el costo de CPU de Python. En un disco real el costo estaría dominado por el número de transferencias.

En el Experimento 1 se mide la *inserción estructural* en todas las organizaciones (sin validar la unicidad de la PK, porque las claves generadas son únicas por construcción): así se compara lo mismo en todas, ya que el Heap sin índice no valida y los índices B⁺/hash se miden por separado. En el motor SQL la unicidad sí se valida (secciones 4 y 5).

9.2. Experimento 1: costo de inserción masiva

Se insertan lotes $N \in \{10^3, 10^4, 5 \cdot 10^4, 10^5, 2.5 \cdot 10^5, 5 \cdot 10^5\}$ con $B = 4096$. Los índices B⁺ y hash son índices únicos sobre `id` de un Heap: se reporta el costo total (heap + índice) y el del índice solo.

Estructura	$N = 1\,000$	$N = 10\,000$	$N = 50\,000$	$N = 100\,000$	$N = 250\,000$	$N = 500\,000$
<i>Tiempo total (s)</i>						
Heap	0.01	0.09	0.48	0.94	2.36	4.79
Sequential sin reorg	0.01	0.13	0.63	1.23	3.28	6.49
Sequential con reorg	0.03	0.32	1.98	4.07	11.74	25.74
B+	0.06	0.56	3.88	8.02	20.53	41.56
Hash	0.03	0.24	1.21	2.37	5.93	11.90
<i>Escrituras de páginas</i>						
Heap	1 001	10 001	50 001	100 001	250 001	500 001
Sequential sin reorg	1 016	10 154	50 770	101 539	253 847	507 693
Sequential con reorg	1 108	10 918	55 077	110 975	273 965	547 805
B+	2 009	20 090	100 516	201 013	502 889	1 005 706
Hash	2 010	20 069	100 263	200 520	502 059	1 004 109
<i>Lecturas de páginas</i>						
Heap	984	9 846	49 230	98 461	246 153	492 307
Sequential sin reorg	1 933	19 933	99 933	199 933	499 933	999 933
Sequential con reorg	3 288	62 843	429 335	958 162	2 723 390	5 946 007
B+	2 576	29 463	148 987	298 383	855 355	1 851 665
Hash	2 989	29 882	149 364	298 724	747 186	1 494 366
<i>I/O por inserción (lecturas + escrituras) / N</i>						
Heap	1.99	1.98	1.98	1.98	1.98	1.98
Sequential sin reorg	2.95	3.01	3.01	3.01	3.02	3.02
Sequential con reorg	4.40	7.38	9.69	10.69	11.99	12.99
B+	4.58	4.96	4.99	4.99	5.43	5.71
Hash	5.00	5.00	4.99	4.99	5.00	5.00
<i>Solo el índice (sin el heap): I/O por inserción</i>						
B+	2.60	2.97	3.01	3.01	3.45	3.73
Hash	3.01	3.01	3.01	3.01	3.01	3.01

Tabla 6: Experimento 1: tiempo, escrituras, lecturas e I/O por inserción ($B = 4096$).

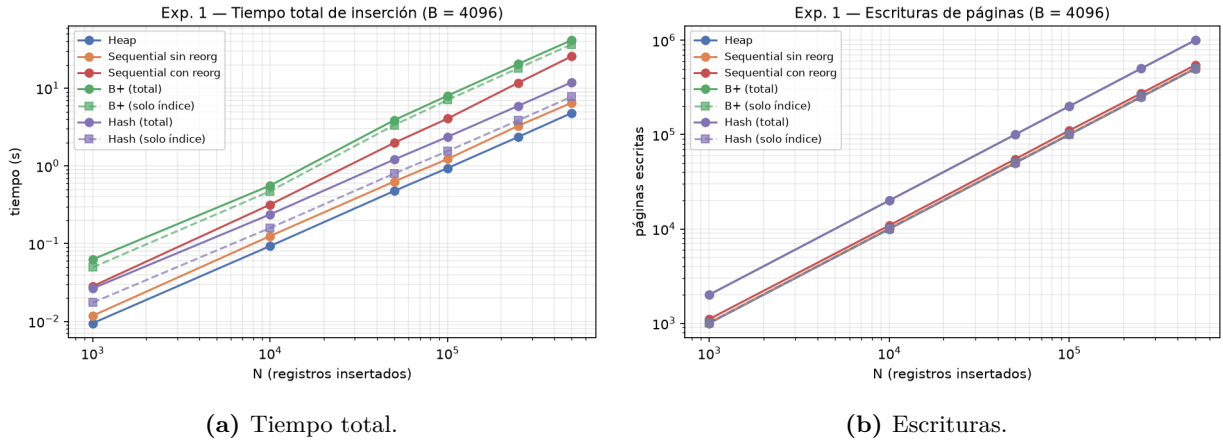


Figura 5: Experimento 1 en escala log-log: todas las curvas son lineales en N (pendiente 1), es decir, costo por inserción aproximadamente constante.

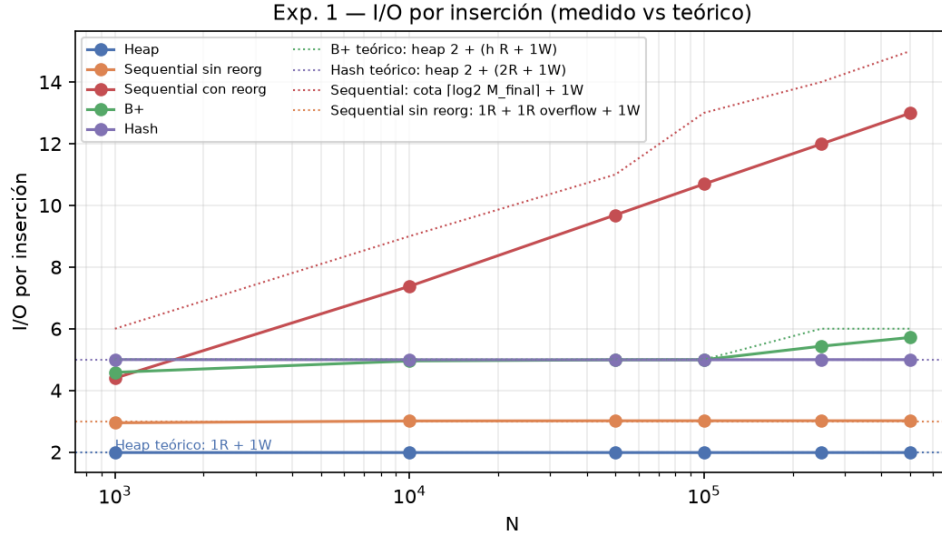


Figura 6: I/O por inserción medido frente al costo teórico.

Análisis.

- **Heap:** 1.98 I/O por inserción para todo N , exactamente el costo teórico de $1R + 1W$ (el 0.02 que falta son las inserciones que abren una página nueva, que cuestan solo $1W$: una de cada 65). Es la inserción más barata y la más rápida (≈ 4.8 s para 500 000 registros).
- **Sequential sin reorganización:** ≈ 3 I/O por inserción ($1R$ de la página principal, $1R + 1W$ de la cabeza de su overflow). La inserción es barata, pero el archivo degenera: con 500 000 registros queda 1 página principal y 7 692 de overflow, y una búsqueda puntual posterior cuesta en promedio 4 026 lecturas (tabla 7, figura 7); es decir, sin reorganizar el Sequential se comporta como un Heap con peor inserción.
- **Sequential con reorganización:** el costo por inserción crece como $\log_2 M$ (4.4 I/O con 1 000 registros, 13.0 con 500 000): cada inserción paga la búsqueda binaria de su página. Las reorganizaciones (35 en total para 500 000 registros) suman solo ≈ 0.1 escrituras por inserción, porque el archivo crece geométricamente entre una y otra (con fill factor 0.75 cada página admite ≈ 35 % más registros antes de desbordar: pasa de 48 a 65). A cambio, al terminar la carga la búsqueda cuesta $\lceil \log_2 M \rceil \approx 13$ lecturas.
- **B⁺:** el índice solo cuesta $hR + 1W$: 3.0 I/O por inserción mientras $h = 2$ y 3.7 con 500 000 claves, cuando la altura pasa a 3 (tabla 8); los splits agregan poco (una hoja se divide cada ≈ 290 inserciones). Es la estructura más lenta en tiempo porque cada inserción decodifica y reescribe nodos de hasta 508 claves.
- **Hash:** 3.0 I/O por inserción del índice para todo N ($2R + 1W$), independiente de N ; la profundidad global crece de 2 a 11 y el directorio ocupa 1 a 3 páginas. Los splits y las duplicaciones del directorio son poco frecuentes, y `crc32` reparte las claves de forma tan uniforme que los buckets quedan casi llenos (96 % de ocupación con 100 000 claves).

N	Sequential sin reorg			Sequential con reorg		
	M	overflow	búsqueda	M	overflow (reorgs)	búsqueda
1 000	1	15	9.5	19	0 (8)	4.4
10 000	1	153	79.9	168	7 (17)	7.4
50 000	1	769	394.3	957	0 (25)	9.9
100 000	1	1 538	742.2	2 064	0 (28)	11.0
250 000	1	3 846	1 742.6	4 509	8 (32)	12.2
500 000	1	7 692	4 026.3	9 045	22 (35)	13.1

Tabla 7: Sequential File tras la carga: páginas principales (M), páginas de overflow y lecturas promedio de una búsqueda puntual posterior (100 claves aleatorias).

Estructura	$N = 1\,000$	$N = 10\,000$	$N = 50\,000$	$N = 100\,000$	$N = 250\,000$	$N = 500\,000$
B^+ : altura h	2	2	2	2	3	3
Hash: profundidad global	2	5	7	8	10	11

Tabla 8: Forma final de los índices en el Experimento 1.

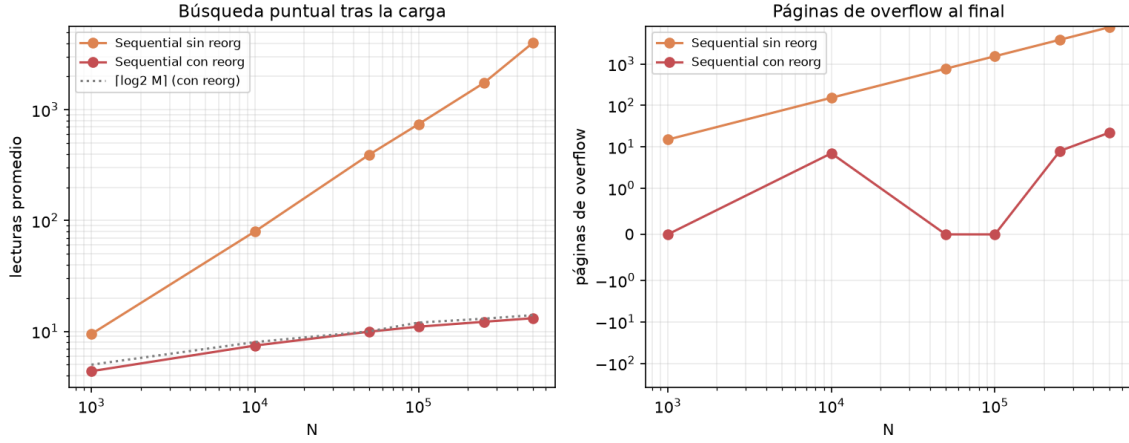


Figura 7: Lo que compra la reorganización: sin ella la búsqueda posterior es lineal en N (la cadena de overflow crece sin límite); con ella se mantiene en $\lceil \log_2 M \rceil$.

9.3. Experimento 2: búsquedas puntuales de igualdad

Con $N = 100\,000$ registros ($B = 4096$) se ejecutan 1 000 búsquedas de claves aleatorias existentes en cada estructura. Las lecturas incluyen traer el registro por RID en los índices.

Método	lecturas (media)	σ	teórico	ms (media)	σ (ms)	parámetros
Full Scan (Heap)	1 539.00	0.00	1 539	22.726	1.453	$P = 1539$ páginas
Búsqueda binaria (Sequential)	11.03	0.18	11.03	0.091	0.151	$M = 2084$ páginas principales
B^+ (IndexScan)	3.00	0.00	3	0.038	0.007	$h = 2$
Hash (IndexScan)	3.00	0.00	3	0.014	0.007	global depth = 8

Tabla 9: Experimento 2: 1 000 búsquedas de igualdad sobre 100 000 registros.

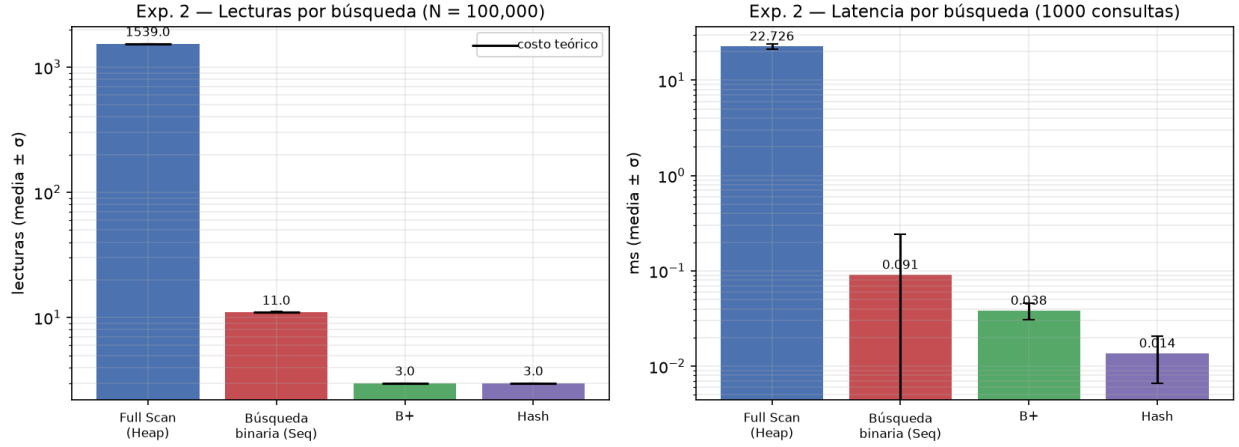


Figura 8: Experimento 2 (escala logarítmica). La raya negra marca el costo teórico.

Análisis. Los cuatro métodos coinciden con la teoría: el **full scan** lee siempre las $P = 1539$ páginas ($\sigma = 0$), la **búsqueda binaria** sobre $M = 2084$ páginas principales hace en promedio $\log_2 M = 11.03$ sondeos ($\lfloor \log_2 M \rfloor$ o $\lceil \log_2 M \rceil$ según la clave, de ahí $\sigma = 0.18$), el **B⁺** de altura $h = 2$ cuesta exactamente $h + 1 = 3$ lecturas y el **hash** exactamente 3 (directorio + bucket + registro, sin overflow). El índice reduce el costo en un factor de ≈ 500 frente al full scan. En tiempo, B⁺ y hash hacen las mismas 3 lecturas pero el hash es casi 3 veces más rápido: busca la clave en el bucket con una búsqueda de bytes en C, mientras que el B⁺ decodifica dos nodos completos con **struct**. La búsqueda binaria es ≈ 250 veces más rápida que el full scan a pesar de no usar índice.

9.4. Experimento 3: búsquedas por rango con selectividad variable

Con $N = 100\,000$ ($B = 4096$) se consultan rangos **id BETWEEN a AND a+k-1** que devuelven el 0.1 %, 1 %, 5 %, 10 % y 25 % de la tabla (10 rangos aleatorios por selectividad).

Selectividad	k	B ⁺			Sequential			Full Scan (Heap)		
		lecturas	teórico	ms	lecturas	teórico	ms	lecturas	teórico	ms
0.1 %	100	102.3	102.0	1.47	13.2	13.1	0.27	1539.0	1539.0	23.98
1 %	1000	1005.5	1005.0	4.04	31.8	31.9	1.58	1539.0	1539.0	25.31
5 %	5000	5018.8	5019.0	20.27	115.2	115.2	7.55	1539.0	1539.0	28.16
10 %	10000	10037.2	10036.0	39.28	219.5	219.4	15.07	1539.0	1539.0	34.45
25 %	25000	25087.8	25087.0	109.18	531.7	531.9	38.19	1539.0	1539.0	48.57

Tabla 10: Experimento 3: lecturas medidas (media de 10 rangos), costo teórico y tiempo en ms. Teórico: B⁺ = $h + \lceil k/f \rceil - 1 + k$; Sequential = $\log_2 M + k/r$ con $r = 48$; full scan = P .

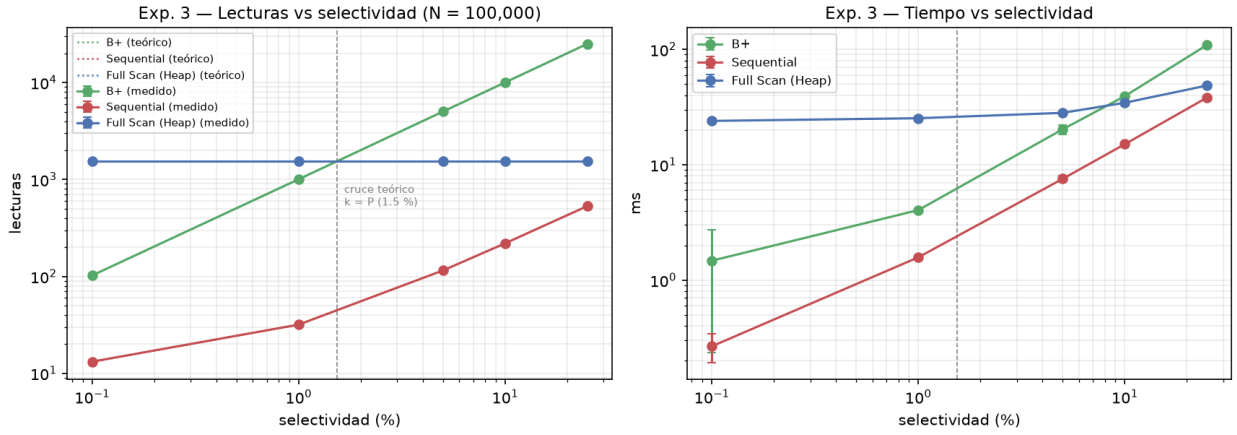


Figura 9: Experimento 3: lecturas y tiempo frente a la selectividad (log-log). La línea vertical marca el cruce teórico entre el B⁺ no agrupado y el full scan ($k \approx P$).

Análisis.

- **B⁺ (no agrupado):** cuesta $\approx k$ lecturas (una por registro, porque cada RID apunta a una página distinta del Heap) más h y las hojas recorridas (1 hoja cada ≈ 290 entradas). Supera al full scan cuando $k > P$, es decir, con selectividad mayor a $P/N = 1.5\%$: con 25 % hace 25 088 lecturas, 16 veces más que el full scan. Este es el cruce que justifica que un optimizador basado en costos (modo `cost` de nuestro planner) elija el full scan para rangos amplios.
- **Sequential (agrupado):** la búsqueda binaria ubica la primera página ($\log_2 M \approx 11$ lecturas) y luego lee páginas contiguas con 48 registros cada una: $k/48$ páginas. Es el mejor método en todas las selectividades (532 lecturas para el 25 %, un tercio del full scan) y el costo medido coincide con la fórmula a la décima.
- **Full scan:** $P = 1\,539$ lecturas constantes; solo crece el tiempo de CPU (filtrar y decodificar más filas).
- **Tiempo vs I/O:** en tiempo el B⁺ le gana al full scan hasta $\approx 5\%$ aunque ya lea 3 veces más páginas, porque sus lecturas aleatorias caen en la caché del sistema operativo y cuestan poco; en un disco real, donde una lectura aleatoria cuesta de 0.1 ms (SSD) a varios ms (HDD), el cruce se acerca al que predice el conteo de I/O (1.5 %).

9.5. Experimento 4: sensibilidad al tamaño de bloque

Para $B \in \{1024, 2048, 4096, 8192\}$ se insertan 100 000 registros en un Heap con un B⁺ único sobre `id` y luego se hacen 1 000 búsquedas puntuales. Como referencia se reporta también el Sequential (reorganizado) y el hash con el mismo B .

B (bytes)	1 024	2 048	4 096	8 192
registros por página del heap	16	32	65	131
páginas del heap (P)	6 250	3 125	1 539	764
entradas por hoja (L)	100	202	407	816
fan-out del interno ($m + 1$)	125	253	509	1 021
altura h medida	3	3	2	2
altura teórica [mín, máx]	[3, 3]	[3, 3]	[2, 3]	[2, 2]
hojas / internos	1 422 / 17	714 / 5	341 / 1	161 / 1
I/O de inserción (heap + B^+)	591 248	563 984	499 396	499 046
de los cuales en el B^+	397 497	367 108	300 934	299 809
lecturas por búsqueda ($h + 1$)	4.00	4.00	3.00	3.00
I/O total (inserción + 1000 búsquedas)	595 248	567 984	502 396	502 046
MB transferidos (I/O $\times B$)	581.3	1 109.3	1 962.5	3 922.2
tiempo de inserción (s)	3.70	5.44	7.53	12.08
Sequential: M / lecturas por búsqueda	8 334 / 13.04	4 167 / 12.03	2 084 / 11.04	1 021 / 10.00
Hash: profundidad global / buckets	10 / 1 024	9 / 512	8 / 256	7 / 128

Tabla 11: Experimento 4: efecto del tamaño de página ($N = 100\,000$). La altura teórica es el intervalo entre nodos llenos y nodos al 50 %.

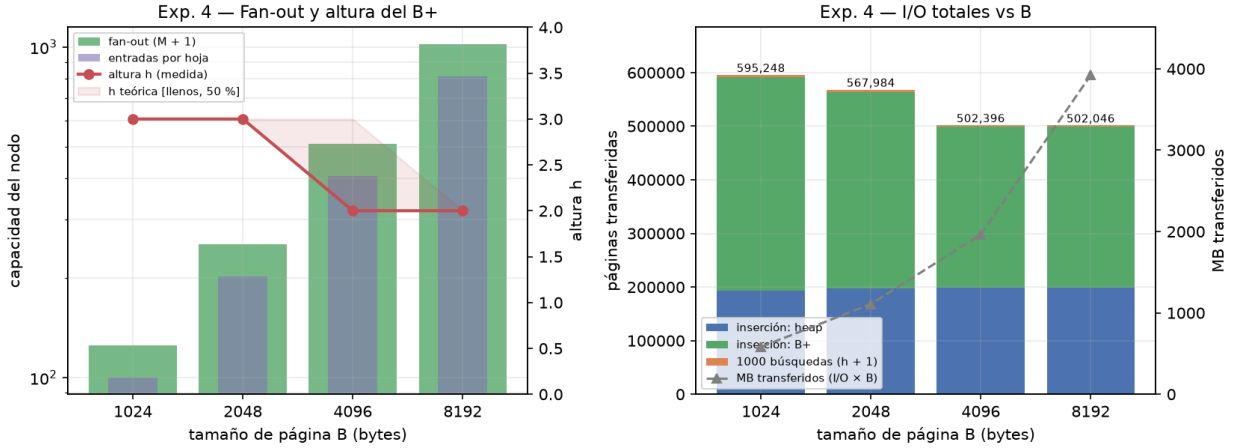


Figura 10: Experimento 4: fan-out y altura del B^+ (izq.) e I/O totales y bytes transferidos (der.).

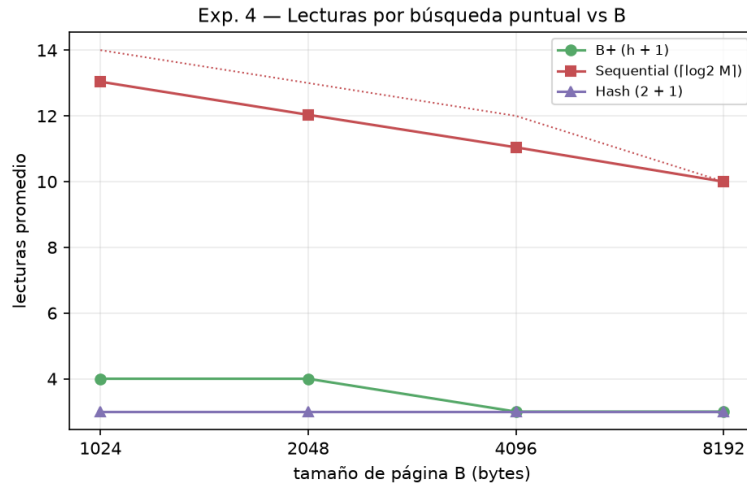


Figura 11: Lecturas por búsqueda puntual según B : $B^+ (h + 1)$, Sequential ($\log_2 M$) y hash (3).

Análisis. El fan-out crece linealmente con B (125, 253, 509 y 1021) y la altura baja de 3 a 2 al pasar de 2048 a 4096 bytes, siempre dentro del intervalo teórico. Como cada búsqueda cuesta $h+1$, pasa de 4 a 3 lecturas, y como cada inserción cuesta $hR + 1W$ en el índice, las I/O de inserción del B^+ bajan de 397 497 a 299 809 (las del Heap, 2 por registro, no dependen de B). El total de I/O baja un 16 % de $B = 1024$ a $B = 8192$, pero el volumen transferido ($I/O \times B$) crece casi linealmente: de 581 MB a 3 922 MB. Duplicar B más allá de 4096 no reduce la altura para 100 000 claves (la raíz ya cabe en una página) y solo encarece cada transferencia: páginas de 4 KB, el tamaño de página del sistema operativo, son el mejor compromiso para este volumen. El Sequential muestra el mismo efecto ($\log_2 M$ baja de 13 a 10 lecturas) y el hash mantiene 3 lecturas con cualquier B , con una profundidad global que baja de 10 a 7 porque los buckets son más grandes.

10. Pruebas

La suite de `pytest` (`tests/`, un archivo por módulo) cubre: round-trip de header, página, registro, RID y metadatos; **exactitud del contador** (inserción en Heap con página libre = $1R + 1W$, búsqueda en $B^+ = h$ lecturas, full scan = P lecturas, hash = 2 lecturas, búsqueda binaria $\leq \lceil \log_2 M \rceil + 1$); invariantes del B^+ tras 20 000 inserciones aleatorias (claves ordenadas en cada nodo, cotas de los separadores, hojas enlazadas en ambos sentidos, todas a la misma profundidad, nodos al menos al 50 % y altura entre las cotas de $\log_M N$); hash con duplicados, profundidad máxima y splits del directorio en varias páginas; **reorganize** (conserva registros, vacía el overflow, respeta el fill factor y usa ordenamiento externo con cadenas largas); persistencia (cerrar, reabrir y obtener los mismos resultados); parser (casos válidos y errores con posición); planner y ejecutor de punta a punta; y la API REST. Además, pruebas aleatorias *diferenciales* ejecutan miles de inserciones, borrados y búsquedas sobre cada estructura (y por SQL, con índices B^+ y hash sobre una tabla Sequential) y comparan cada resultado con un modelo en memoria, con páginas de 512 bytes para forzar splits, overflow, reorganizaciones y cierres/reaperturas. En total son 133 pruebas que corren en ≈ 12 s.

11. Conclusiones

- Contar las páginas transferidas (y no solo el tiempo) permite verificar la teoría con exactitud: los costos medidos coinciden con las fórmulas de la tabla 5 en todos los experimentos.
- El Heap tiene la inserción más barata (2 I/O) pero toda búsqueda sin índice cuesta P .
- El Sequential File ofrece búsquedas en $\lceil \log_2 M \rceil$ y los rangos más baratos por estar agrupado, a cambio de reorganizar periódicamente: sin reorganización la inserción es barata pero el archivo degenera en una cadena de overflow y la búsqueda se vuelve lineal.
- El B^+ da búsquedas en $h+1$ con h muy baja (2–3 para 100 000 claves) y soporta rangos, pero como índice no agrupado pierde frente al full scan cuando la selectividad supera $\approx P/N$; ahí el modo *cost* del planificador elige el full scan.
- El hash extensible logra búsquedas en 3 lecturas independientes de N , sin soportar rangos.
- Páginas más grandes aumentan el fan-out y reducen la altura y el número de I/O, pero cada I/O transfiere más bytes: el volumen transferido crece casi linealmente con B .

Trabajo futuro. Buffer pool con política LRU (desactivable para medir), fusión y redistribución en el B^+ , recorrido de índice ordenado por página (*bitmap heap scan*), transacciones con WAL y, para el Entregable 2, los módulos espacial (R-Tree), textual (SPIMI/BM25) y vectorial (IVF/HNSW).

Referencias

- [1] R. Ramakrishnan y J. Gehrke. *Database Management Systems*, 3.^a ed. McGraw-Hill, 2003.
- [2] H. Garcia-Molina, J. D. Ullman y J. Widom. *Database Systems: The Complete Book*, 2.^a ed. Pearson, 2008.
- [3] A. Silberschatz, H. F. Korth y S. Sudarshan. *Database System Concepts*, 7.^a ed. McGraw-Hill, 2019.
- [4] R. Fagin, J. Nievergelt, N. Pippenger y H. R. Strong. Extendible hashing — a fast access method for dynamic files. *ACM TODS*, 4(3):315–344, 1979.
- [5] D. Comer. The ubiquitous B-tree. *ACM Computing Surveys*, 11(2):121–137, 1979.
- [6] P. G. Selinger et al. Access path selection in a relational database management system. *SIGMOD*, 1979.